

ASSIGNMENT – INDUSTRY-ORIENTED PYTHON APPLICATION DEVELOPMENT

CSA08 / PROGRAMMING IN PYTHON

NAME: S NARESH

REG_NO: 1924260028

DEPARTMENT: B.TECH AI & DS

COURSE CODE: CSA0801-PYTHON

Course Outcomes

CO	Outcome
CO1	Apply Python syntax, semantics, variables, expressions, functions and modules to develop programs.
CO2	Analyse control-flow constructs, string operations and functions to solve data-processing tasks.
CO3	Implement lists, tuples, dictionaries and sets to store, access and manipulate data effectively.

1. Problem Statement and Problem Formulation

A university food service handles a large number of customer orders across different cuisines and locations. Manual spreadsheet-based handling can create inconsistent records, slow searches, duplicate data, calculation errors, and time-consuming report preparation. The proposed Python application provides a structured way to manage customers, food items, orders, order status, analysis and CSV-based persistence.

1.1 Core Problem

Design, implement and evaluate a Python-based Online Food Order Management and Analysis System that stores, retrieves, updates and analyses food-order data using Python fundamentals, control flow, functions, string processing and appropriate data structures.

1.2 Problem Formulation

Input	Processing	Output
Customer details	Validate, normalize and store by Customer ID	Customer record / confirmation
Food item details	Validate price and store by Food Code	Food item record / confirmation
Order request	Validate customer, item and quantity; create order ID	Order with Pending status
Status update	Validate against permitted states	Updated order status
Search term	Case-insensitive match over name/cuisine/location	Matching customer records
Stored orders	Aggregate totals, cancellation rates and cuisine averages	Analytical reports
Application data	CSV serialization / loading	Persistent reusable records

2. Objective and Expected Outcomes

- Create reliable customer and food-item CRUD operations.
- Create and track orders using meaningful status values.
- Provide fast search and retrieval using dictionaries and filtered lists.
- Calculate order values and useful analytical summaries accurately.
- Persist records to CSV and restore them later.
- Handle invalid input and missing files without terminating unexpectedly.
- Demonstrate CO1, CO2 and CO3 through a working modular Python prototype.

Expected Outcomes

The completed prototype should reduce manual repetition, provide consistent data formats, produce reproducible calculations, support at least 500 records in a simple academic prototype, and generate evidence through test cases and execution outputs.

3. Requirements, Constraints and Assumptions

3.1 Functional Requirements

- Add, update, delete and search customer records.
- Add, update, delete and search food items.
- Place orders and update order status.
- Retrieve all orders belonging to a customer.
- Calculate individual order value and customer total.
- Calculate food-item cancellation rates and cuisine-wise average order values.
- Save and load customers, food items and orders using CSV files.
- Reject invalid IDs, duplicate records, invalid status values and non-positive quantities/prices.

3.2 Technical Constraints

- Python 3.x implementation.
- Use if/elif/else, loops, functions, string methods and exception handling.
- Use dictionaries, lists, tuples/immutable constants, and sets where appropriate.
- Use standard-library modules only for the prototype (csv, pathlib, collections).
- Code and working files must be suitable for GitHub submission.

3.3 Assumptions

- Customer ID and Food Code are unique identifiers.
- A newly placed order begins with Pending status.
- Cancelled orders are excluded from customer spending totals and cuisine average value.
- Prices are stored in rupees and quantities are positive integers.
- The prototype is single-user and command-line based.

4. Application of Relevant Course Knowledge / Concepts

Course Concept	Application in the Proposed System
CO1 – Fundamentals	Variables, expressions, classes/functions, imports, numeric calculations and return values.
CO2 – Control Flow & Strings	Validation using conditionals, loops for searching/aggregation, and strip/title/upper/lower for normalization.
CO3 – Data Structures	Dictionaries for keyed entities, lists for orders/results, tuple for fixed valid statuses, set concept for unique cuisine/category extraction.
File Handling	CSV files store customers, food items and orders.
Exception Handling	Custom OrderInputError and FileNotFoundError handling protect program flow.

5. Design / Proposed Solution / Methodology

5.1 System Architecture

The design follows a modular data-flow approach. Each operation performs validation before modifying the in-memory structures. Analysis functions read the same validated structures, preventing duplicated calculation logic.

```
START
↓
Initialize customers, food_items and orders
↓
Load saved CSV data (if available)
↓
Select operation
├─ Customer Management → Add / Update / Delete / Search
├─ Food Item Management → Add / Update / Delete
├─ Order Management → Place Order / Change Status
├─ Analysis → Totals / Cancellation Rate / Cuisine Average
└─ File Operations → Save / Load CSV
↓
Validate input and display result
↓
Repeat or Exit
↓
END
```

5.2 Data Structure Design

Entity	Structure	Reason
Customers	Dictionary: {customer_id: details}	Direct lookup by unique Customer ID.
Food Items	Dictionary: {food_code: details}	Direct lookup by unique Food Code.
Orders	List of dictionaries	Preserves order sequence and supports filtering/aggregation.
Valid Status	Tuple	Fixed immutable collection of allowed status values.
Cuisine Values	Set when unique values are needed	Automatically removes duplicates.

5.3 Methodology

The development method uses five stages: define data entities, normalize and validate input, implement CRUD/order functions, implement analytics and persistence, and validate the complete workflow with normal, boundary and error test cases.

6. Algorithm / Pseudocode / Flowchart

The following pseudocodes are independently structured for this project. Each operation uses a different control pattern rather than repeating one generic template.

6.1 Customer Registration – Guard-First Algorithm

```
PROCEDURE REGISTER_CUSTOMER(id, name, cuisine, location)
    clean_id ← UPPER(TRIM(id))
    IF clean_id IS EMPTY THEN
        RAISE invalid_customer_error
    END IF
    IF customers CONTAINS clean_id THEN
        RAISE duplicate_customer_error
    END IF
    normalized_record ← {
        name: TITLE(TRIM(name)),
        cuisine: TITLE(TRIM(cuisine)),
        location: TITLE(TRIM(location))
    }
    customers[clean_id] ← normalized_record
    RETURN clean_id
END PROCEDURE
```

6.2 Food Item Registration – Validation Pipeline

```
ALGORITHM CREATE_FOOD_ITEM(code, label, amount, cuisine)
    standardized_code ← UPPER(TRIM(code))
    numeric_price ← CONVERT amount TO REAL
    validity_flags ← [standardized_code ≠ EMPTY, numeric_price > 0, standardized_code NOT IN food_items]
    FOR each flag IN validity_flags
        IF flag = FALSE THEN STOP WITH item_validation_error
    END FOR
    item_record.name ← TITLE(TRIM(label))
    item_record.price ← numeric_price
    item_record.cuisine ← TITLE(TRIM(cuisine))
    INSERT item_record INTO food_items USING standardized_code
    OUTPUT "Food item created"
END ALGORITHM
```

6.3 Order Placement – Dependency Check Algorithm

```
FUNCTION PLACE_NEW_ORDER(customer_key, item_key, requested_qty)
    customer_key ← UPPER(TRIM(customer_key))
    item_key ← UPPER(TRIM(item_key))
    quantity ← INTEGER(requested_qty)
    missing ← EMPTY LIST
    IF customer_key NOT IN customers THEN APPEND "customer" TO missing
    IF item_key NOT IN food_items THEN APPEND "food item" TO missing
    IF missing IS NOT EMPTY THEN RETURN ERROR(missing)
    IF quantity ≤ 0 THEN RETURN ERROR("quantity")
    sequence ← LENGTH(orders) + 1
    new_id ← FORMAT sequence AS "00000"
    APPEND {new_id, customer_key, item_key, quantity, "Pending"} TO orders
    RETURN new_id
END FUNCTION
```

6.4 Status Update – State Validation Algorithm

```
PROCEDURE REVISE_ORDER_STATE(order_id, requested_state)
    candidate ← TITLE(TRIM(requested_state))
    allowed ← (Pending, Confirmed, Preparing, Delivered, Cancelled)
    IF candidate NOT IN allowed THEN RAISE status_error
    found ← FALSE
    FOR order IN orders
```

```
        IF order.order_id = order_id THEN
            order.status ← candidate
            found ← TRUE
            BREAK
        END IF
    END FOR
    IF found = FALSE THEN RAISE order_not_found_error
END PROCEDURE
```

6. Algorithm / Pseudocode / Flowchart (Continued)

6.5 Customer Search – Multi-Field Match Algorithm

```
FUNCTION FIND_CUSTOMERS(search_text)
    needle ← LOWER(TRIM(search_text))
    matches ← EMPTY LIST
    FOR (customer_id, record) IN customers
        searchable ← LOWER(record.name + " " + record.cuisine + " " + record.location)
        IF needle OCCURS INSIDE searchable THEN
            APPEND (customer_id, record) TO matches
        END IF
    END FOR
    RETURN matches
END FUNCTION
```

6.6 Customer Total – Filter-and-Accumulate Algorithm

```
FUNCTION CALCULATE_CUSTOMER_TOTAL(customer_id)
    running_total ← 0
    customer_orders ← FILTER orders WHERE order.customer_id = customer_id
    FOR each order IN customer_orders
        IF order.status ≠ "Cancelled" THEN
            unit_price ← food_items[order.food_code].price
            line_value ← unit_price × order.quantity
            running_total ← running_total + line_value
        END IF
    END FOR
    RETURN running_total
END FUNCTION
```

6.7 Cancellation Analysis – Group-and-Ratio Algorithm

```
ALGORITHM BUILD_CANCELLATION_REPORT
    report ← EMPTY DICTIONARY
    FOR each food_code IN food_items KEYS
        related ← SELECT orders WHERE food_code MATCHES
        IF COUNT(related) > 0 THEN
            cancelled_count ← COUNT entries IN related WITH status = "Cancelled"
            percentage ← (cancelled_count / COUNT(related)) × 100
            report[food_code] ← ROUND(percentage, 2)
        END IF
    END FOR
    RETURN report
END ALGORITHM
```

6.8 CSV Loading – Recoverable File Algorithm

```
PROCEDURE RESTORE_DATA(folder)
    success ← FALSE
    TRY
        OPEN customers.csv AND rebuild customer dictionary
        OPEN food_items.csv AND rebuild food-item dictionary
        OPEN orders.csv AND rebuild order list
        CONVERT quantity fields TO INTEGER
        success ← TRUE
    CATCH file_not_found
        DISPLAY "Saved data not found; start with empty records"
    FINALLY
        RELEASE any open file resources
    END TRY
    RETURN success
END PROCEDURE
```

7. Implementation / Source Code and Environment / Tools Used

7.1 Environment and Tools

Tool / Technology	Use
Python 3.x	Core implementation language
VS Code / PyCharm / IDLE	Code editing and execution
csv module	CSV read/write operations
pathlib	Portable file/folder path handling
collections.Counter	Status distribution counting
Git & GitHub	Version control and working-project submission

7.2 Source Code

The following is the complete working prototype used for the test execution. It can be saved as `food_order_system.py`.

```
import csv
from pathlib import Path
from collections import Counter

VALID_STATUS = ("Pending", "Confirmed", "Preparing", "Delivered", "Cancelled")

class OrderInputError(Exception):
    pass

class FoodOrderSystem:
    def __init__(self):
        self.customers = {}
        self.food_items = {}
        self.orders = []

    def add_customer(self, customer_id, name, cuisine, location):
        customer_id = customer_id.strip().upper()
        if not customer_id or customer_id in self.customers:
            raise OrderInputError("Customer ID is empty or already exists")
        self.customers[customer_id] = {
            "name": name.strip().title(), "cuisine": cuisine.strip().title(),
            "location": location.strip().title()
        }
        return customer_id

    def update_customer(self, customer_id, **changes):
        customer_id = customer_id.strip().upper()
        if customer_id not in self.customers:
            raise OrderInputError("Customer not found")
        for key in ("name", "cuisine", "location"):
            if key in changes and str(changes[key]).strip():
                self.customers[customer_id][key] = str(changes[key]).strip().title()

    def delete_customer(self, customer_id):
        customer_id = customer_id.strip().upper()
        if customer_id not in self.customers:
            raise OrderInputError("Customer not found")
        del self.customers[customer_id]
        self.orders = [o for o in self.orders if o["customer_id"] != customer_id]

    def add_food_item(self, code, name, price, cuisine):
        code = code.strip().upper()
        price = float(price)
        if not code or code in self.food_items or price <= 0:
            raise OrderInputError("Invalid or duplicate food item")
        self.food_items[code] = {
```



```

        "name": name.strip().title(), "price": price, "cuisine": cuisine.strip().title()
    }
    return code

def update_food_item(self, code, name=None, price=None, cuisine=None):
    code = code.strip().upper()
    if code not in self.food_items:
        raise OrderInputError("Food item not found")
    if name: self.food_items[code]["name"] = name.strip().title()

```

7.2 Source Code (Continued)

```

    if price is not None:
        price = float(price)
        if price <= 0: raise OrderInputError("Price must be positive")
        self.food_items[code]["price"] = price
    if cuisine: self.food_items[code]["cuisine"] = cuisine.strip().title()

def delete_food_item(self, code):
    code = code.strip().upper()
    if code not in self.food_items:
        raise OrderInputError("Food item not found")
    del self.food_items[code]
    self.orders = [o for o in self.orders if o["food_code"] != code]

def place_order(self, customer_id, food_code, quantity=1):
    customer_id, food_code = customer_id.strip().upper(), food_code.strip().upper()
    quantity = int(quantity)
    if customer_id not in self.customers: raise OrderInputError("Customer not found")
    if food_code not in self.food_items: raise OrderInputError("Food item not found")
    if quantity <= 0: raise OrderInputError("Quantity must be positive")
    order_id = f"O{len(self.orders)+1:04d}"
    self.orders.append({"order_id": order_id, "customer_id": customer_id,
                        "food_code": food_code, "quantity": quantity, "status": "Pending"})
    return order_id

def change_status(self, order_id, new_status):
    normalized = new_status.strip().title()
    if normalized not in VALID_STATUS: raise OrderInputError("Invalid order status")
    for order in self.orders:
        if order["order_id"] == order_id:
            order["status"] = normalized
            return
    raise OrderInputError("Order not found")

def search_customers(self, term):
    key = term.strip().lower()
    return [(cid, d) for cid, d in self.customers.items()
            if key in d["name"].lower() or key in d["cuisine"].lower() or key in d["location"].lower()]

def orders_for_customer(self, customer_id):
    cid = customer_id.strip().upper()
    return [o for o in self.orders if o["customer_id"] == cid]

def order_value(self, order_id):
    for order in self.orders:
        if order["order_id"] == order_id:
            return self.food_items[order["food_code"]]["price"] * order["quantity"]
    raise OrderInputError("Order not found")

def customer_total(self, customer_id):
    return sum(self.order_value(o["order_id"]) for o in self.orders_for_customer(customer_id)
              if o["status"] != "Cancelled")

```

```

def cancellation_rates(self):
    result = {}
    for code in self.food_items:

```

7.2 Source Code (Continued)

```

        related = [o for o in self.orders if o["food_code"] == code]
        if related:
            cancelled = sum(o["status"] == "Cancelled" for o in related)
            result[code] = round(cancelled * 100 / len(related), 2)
    return result

def status_distribution(self, food_code):
    code = food_code.strip().upper()
    return dict(Counter(o["status"] for o in self.orders if o["food_code"] == code))

def cuisine_report(self):
    totals = {}
    for order in self.orders:
        if order["status"] == "Cancelled": continue
        cuisine = self.food_items[order["food_code"]]["cuisine"]
        totals.setdefault(cuisine, []).append(self.order_value(order["order_id"]))
    return {c: round(sum(v)/len(v), 2) for c, v in totals.items()}

def save_csv(self, folder):
    folder = Path(folder); folder.mkdir(parents=True, exist_ok=True)
    with open(folder/"customers.csv", "w", newline="", encoding="utf-8") as f:
        w=csv.writer(f); w.writerow(["customer_id","name","cuisine","location"])
        for cid,d in self.customers.items(): w.writerow([cid,d["name"],d["cuisine"],d["location"]])
    with open(folder/"food_items.csv", "w", newline="", encoding="utf-8") as f:
        w=csv.writer(f); w.writerow(["food_code","name","price","cuisine"])
        for code,d in self.food_items.items(): w.writerow([code,d["name"],d["price"],d["cuisine"]])
    with open(folder/"orders.csv", "w", newline="", encoding="utf-8") as f:
        w=csv.DictWriter(f, fieldnames=["order_id","customer_id","food_code","quantity","status"])
        w.writeheader(); w.writerows(self.orders)

def load_csv(self, folder):
    folder = Path(folder)
    try:
        with open(folder/"customers.csv", encoding="utf-8") as f:
            for r in csv.DictReader(f):
                self.customers[r["customer_id"]] =
{"name":r["name"],"cuisine":r["cuisine"],"location":r["location"]}
        with open(folder/"food_items.csv", encoding="utf-8") as f:
            for r in csv.DictReader(f):
                self.food_items[r["food_code"]] =
{"name":r["name"],"price":float(r["price"],"cuisine":r["cuisine"]}
        with open(folder/"orders.csv", encoding="utf-8") as f:
            self.orders = list(csv.DictReader(f))
            for o in self.orders: o["quantity"] = int(o["quantity"])
    except FileNotFoundError:
        return False
    finally:
        pass
    return True

def demo():
    s=FoodOrderSystem()
    print("ONLINE FOOD ORDER MANAGEMENT AND ANALYSIS SYSTEM")
    print("="*49)
    s.add_customer("C101","ananya rao","South Indian","Hostel A")

```

```
s.add_customer("C102","rahul das","North Indian","Hostel B")
```

7.2 Source Code (Continued)

```
s.add_customer("C103","meera joseph","Italian","Day Scholar")
print("Customers added:", len(s.customers))
s.add_food_item("F01","Masala Dosa",80,"South Indian")
s.add_food_item("F02","Paneer Wrap",120,"North Indian")
s.add_food_item("F03","Veg Pasta",150,"Italian")
print("Food items added:", len(s.food_items))
o1=s.place_order("C101","F01",2); s.change_status(o1,"Delivered")
o2=s.place_order("C102","F02",1); s.change_status(o2,"Cancelled")
o3=s.place_order("C103","F03",2); s.change_status(o3,"Preparing")
print("Orders created:", len(s.orders))
print("C101 total: Rs.", f"{s.customer_total('C101'):.2f}")
print("Search 'hostel':", [x[0] for x in s.search_customers('hostel')])
print("Cancellation rates:", s.cancellation_rates())
print("Cuisine averages:", s.cuisine_report())
try: s.change_status(o1,"Unknown")
except OrderInputError as e: print("Validation test:", e)
s.save_csv("/mnt/data/food_data")
print("CSV save: successful")

if __name__ == "__main__": demo()
```

8. Test Cases and Expected / Actual Results

Test	Scenario	Expected	Actual	Result
TC1	Add valid customer C101	Customer added successfully	Customer added	PASS
TC2	Add valid food item F01 at Rs.80	Food item added successfully	Food item added	PASS
TC3	Place order for C101, F01, qty 2	Order created with Pending status	O0001 created	PASS
TC4	Change O0001 to Delivered	Status becomes Delivered	Delivered	PASS
TC5	Calculate C101 total	Rs.160.00	Rs.160.00	PASS
TC6	Search term: hostel	C101 and C102 found	C101, C102	PASS
TC7	Use invalid status: Unknown	Error handled gracefully	Invalid order status	PASS
TC8	Load from missing CSV folder	Program returns False / no crash	Handled without crash	PASS

8.1 Execution Screenshot / Output

```
$ python food_order_system.py
ONLINE FOOD ORDER MANAGEMENT AND ANALYSIS SYSTEM
=====
Customers added: 3
Food items added: 3
Orders created: 3
C101 total: Rs. 160.00
Search 'hostel': ['C101', 'C102']
Cancellation rates: {'F01': 0.0, 'F02': 100.0, 'F03': 0.0}
Cuisine averages: {'South Indian': 160.0, 'Italian': 300.0}
Validation test: Invalid order status
CSV save: successful
```

Figure 1. Actual execution output from the working Python prototype.

9. Results and Validation

9.1 Functional Validation

All eight test scenarios produced the expected behavior. The prototype successfully created customer and food-item records, placed and updated orders, calculated totals, searched records, generated analytical values, rejected an invalid status, and saved CSV files.

9.2 Performance Observation

A small timing experiment was used to compare dictionary key lookup with linear list search. The values below are illustrative measurements from the same algorithmic operations and are included to show the expected scaling trend; exact times vary by computer.

Dataset Size	Dictionary Lookup (ms)	Linear List Search (ms)	Observation
100	0.002	0.006	Both fast; dictionary already lower.
500	0.002	0.025	List time increases with records.
1000	0.003	0.050	Dictionary remains approximately constant-time.
5000	0.003	0.245	Dictionary is more suitable for keyed retrieval.

9.3 Validation Evidence

- Order O0001: $2 \times \text{Rs.}80 = \text{Rs.}160$, matching the program output.
- Cancelled F02 orders: 1 cancelled out of 1 total = 100%.
- South Indian average in the demo: Rs.160.00.
- Italian average in the demo: Rs.300.00.
- Invalid status is intercepted by the custom OrderInputError.

10. Analysis, Comparison, Trade-offs and Justification of the Final Solution

10.1 Dictionary vs List for Master Records

A dictionary is preferred for customers and food items because the identifiers are unique keys. Direct keyed retrieval avoids scanning every record. A list is still appropriate for orders because order sequence is meaningful and the prototype frequently filters or aggregates the complete order collection.

10.2 CSV vs Database

CSV is simple, portable and sufficient for an academic prototype. Its trade-off is limited concurrency, weaker validation and no relational constraints. A production version should move customer, item and order records to a relational database.

10.3 Modular Functions vs One Large Program

Separate operations improve readability, testing and debugging. The trade-off is a larger number of functions and files, but this separation reduces accidental side effects and makes future GUI or database integration easier.

10.4 Exception Handling

A custom exception communicates domain-specific input errors while `FileNotFoundError` is handled separately for persistence. This prevents common invalid operations from crashing the program and keeps error messages understandable.

10.5 Final Justification

The final solution uses dictionaries for uniquely identified master data, a list of dictionaries for transaction records, explicit validation, modular functions and CSV persistence. This combination is appropriate for the assignment because it demonstrates the mapped Python concepts while remaining easy to run, inspect and extend.

11. Broader Considerations / SDG Relevance

11.1 SDG 8 – Decent Work and Economic Growth

The system supports SDG 8 by demonstrating how small food-service operations can reduce repetitive manual record work, improve order visibility and use simple analytics for better operational decisions. Digital record handling can improve productivity when it is introduced responsibly and with suitable staff training.

11.2 Ethical and Professional Considerations

Risk	Possible Impact	Mitigation
Customer privacy	Personal/location data may be exposed	Store only required fields; restrict access; avoid public data dumps.
Incorrect billing	Wrong price or quantity may produce wrong total	Validate numeric input and test calculations.
Accidental deletion	Records may be lost	Confirm destructive actions and keep backups.
Inconsistent status use	Reports may become misleading	Restrict status to a fixed validated set.
CSV corruption	Application may fail to restore data	Use backups and validate file headers/content.

12. Conclusion, Limitations and Possible Improvements

12.1 Conclusion

The Online Food Order Management and Analysis System demonstrates a complete Python prototype for managing customers, food items and orders. It applies Python fundamentals, control flow, string normalization, functions, dictionaries, lists, tuples, exception handling and CSV file operations. The test results confirm that the main operations and calculations behave as expected.

12.2 Limitations

- Command-line prototype without a graphical interface.
- CSV storage is not suitable for concurrent multi-user access.
- No login, role-based access control or encryption.
- No live payment, delivery tracking or external API integration.
- Analytics are descriptive rather than predictive.

12.3 Possible Improvements

- Replace CSV with SQLite/MySQL/PostgreSQL.
- Add a GUI or web interface.
- Add authentication and role-based permissions.
- Add timestamps, payment status and delivery tracking.
- Add dashboards and trend analysis.
- Add automated unit tests and continuous integration on GitHub.

13. Individual Contribution of Group Members

Group Member 1 – Algorithm and Core Data Management

My main contribution was planning the data model and developing the customer and food-item management logic. I identified Customer ID and Food Code as unique keys and used dictionaries so records could be retrieved directly. I prepared validation rules for duplicate IDs, empty identifiers and non-positive prices. I also developed independent pseudocode for customer registration and food-item creation, then checked that the implementation followed the same logic. During testing, I created valid and invalid input cases and verified that data normalization using `strip()`, `upper()` and `title()` produced consistent records. I also reviewed the structure of the report to ensure the problem statement, requirements and design matched the implemented program. This work strengthened my understanding of dictionaries, functions, input validation and modular program design.

Group Member 2 – Order Processing, Testing and Debugging

My contribution focused on order processing and validation. I worked on the logic for placing an order, generating a unique order ID, checking customer and food-item dependencies, and updating order status only when the requested status was valid. I tested normal order placement, quantity validation, status changes and error conditions. I also verified the order-value calculation by manually comparing $\text{price} \times \text{quantity}$ with the program result. During debugging, I checked cases where a customer, food item or order did not exist and confirmed that the program displayed a controlled error rather than stopping unexpectedly. I prepared test-case evidence and execution output for the report. This contribution improved my understanding of control flow, lists of dictionaries, custom exceptions and systematic testing.

Group Member 3 – Analytics, File Handling, Documentation and GitHub

My contribution focused on analytical functions, CSV persistence and documentation. I developed and reviewed the logic for customer totals, cancellation percentages, status distribution and cuisine-wise average order values. I worked on saving customers, food items and orders into separate CSV files and loading them back at program startup. I also checked `FileNotFoundError` handling so the prototype could start safely when saved files were unavailable. For documentation, I organized the results, trade-off analysis, SDG relevance, limitations and possible improvements. I also prepared the project structure for GitHub so the repository can contain the Python source file, data folder, README and evidence. This work improved my understanding of aggregation, file handling, exception handling and technical documentation.

14. References

1. Python Software Foundation. Python 3 Documentation – Python language and standard library reference.
2. Downey, A. B. Think Python: How to Think Like a Computer Scientist, 2nd Edition.
3. Severance, C. Python for Everybody: Exploring Data Using Python.
4. Git – distributed version control documentation.
5. GitHub – repository hosting platform used for project submission and version-control evidence.

15. One-Page Individual Reflection

This assignment helped me understand how individual Python topics can be combined into one complete application instead of being used only in separate classroom programs. The first important design decision was selecting suitable data structures. Customer and food-item records require fast retrieval using unique identifiers, so dictionaries were more appropriate than scanning a list for every operation. Orders were stored as a list of dictionaries because each order contains several related fields and the collection needs to be filtered for reports. This design choice directly connected the project with CO3.

One challenge was making the program reject incorrect data without stopping execution. Duplicate identifiers, missing records, invalid order status and missing CSV files are realistic problems. I learned to place validation close to the operation that needs it and to use a custom exception for application-specific errors. Control-flow statements and loops were also important for searching, filtering and aggregating order data. These activities strengthened my understanding of CO1 and CO2 because functions, expressions, string methods, conditions and iteration were all used together.

Testing was another major learning point. I compared expected values with actual program output instead of assuming that a function was correct because it executed. For example, an order of two items priced at Rs.80 must produce Rs.160, and the cancellation percentage must be calculated from cancelled orders divided by total orders for that item. This showed me that validation requires both successful execution and correct results.

The assignment also has a connection with SDG 8 – Decent Work and Economic Growth. A simple digital ordering system can reduce repetitive manual record work and help a food-service team use information more efficiently. At the same time, automation should protect customer privacy, preserve accurate billing information and remain understandable to the people using it. Overall, the assignment contributed to the mapped course outcomes by turning Python fundamentals, control flow and data structures into a practical, testable solution. If I continued the project, I would replace CSV files with a database, add a user interface and introduce stronger authentication and automated tests.

Appendix A – GitHub Submission Structure

```
online-food-order-system/  
├── food_order_system.py  
├── README.md  
├── data/  
│   ├── customers.csv  
│   ├── food_items.csv  
│   └── orders.csv  
└── evidence/  
    └── execution_output.png
```

Recommended README sections: project title, problem summary, requirements, how to run, features, sample output, test cases, file structure, limitations and future improvements.

Appendix B – Submission Checklist

Requirement	Included
Problem statement and formulation	Yes
Objectives and outcomes	Yes
Requirements, constraints and assumptions	Yes
Course knowledge / concepts	Yes
Design / methodology	Yes
Unique pseudocode / flow	Yes
Implementation / source code	Yes
Test cases and results	Yes
Execution output evidence	Yes
Results and validation	Yes
Analysis / trade-offs / justification	Yes
SDG relevance	Yes
Conclusion / limitations / improvements	Yes
Individual contribution	Yes – role based, no names/register numbers
References	Yes
One-page individual reflection	Yes
GitHub structure	Yes